

SOLID PRINCIPLES

Foundations of Object-Oriented Software Design

with Java examples

Prepared by: Ahmed Mandour

Faculty of Computers and Information Sciences — Mansoura University
Java Backend Development

Table of Contents

1. Introduction
2. S — Single Responsibility Principle
3. O — Open/Closed Principle
4. L — Liskov Substitution Principle
5. I — Interface Segregation Principle
6. D — Dependency Inversion Principle
7. Conclusion
8. References

1. Introduction

SOLID is an acronym for five design principles introduced to make object-oriented software easier to understand, maintain, and extend. The term was popularized by Robert C. Martin ("Uncle Bob") and is now considered a foundation of clean software architecture. Each letter in SOLID represents one principle:

- S — Single Responsibility Principle (SRP)
- O — Open/Closed Principle (OCP)
- L — Liskov Substitution Principle (LSP)
- I — Interface Segregation Principle (ISP)
- D — Dependency Inversion Principle (DIP)

Applying these principles reduces tight coupling between classes, makes code easier to test, and allows new features to be added with minimal risk of breaking existing functionality. They are especially relevant in frameworks such as Spring Boot, whose dependency-injection container is built directly on the Dependency Inversion Principle.

S — Single Responsibility Principle

A class should have only one reason to change — that is, it should be responsible for a single, well-defined piece of functionality. Mixing unrelated responsibilities in one class makes it harder to understand, test, and modify safely.

Violation — the class both computes salary and handles persistence:

```
class Employee {  
    double calculateSalary() { ... }  
    void saveToDatabase() { ... } // not this class's job  
}
```

Refactored — responsibilities are separated:

```
class Employee {  
    double calculateSalary() { ... }  
}  
  
class EmployeeRepository {  
    void save(Employee e) { ... }  
}
```

O — Open/Closed Principle

Software entities should be open for extension but closed for modification. New behaviour should be added by writing new code (e.g. a new class), not by editing and risking existing, already-tested code.

Applied with a strategy interface:

```
interface DiscountStrategy {  
    double applyDiscount(double price);  
}
```

```

class StudentDiscount implements DiscountStrategy {
    public double applyDiscount(double price) { return price * 0.9; }
}

class VipDiscount implements DiscountStrategy {
    public double applyDiscount(double price) { return price * 0.7; }
}
// A new discount type = a new class, no existing code is touched

```

L — Liskov Substitution Principle

Objects of a subclass should be replaceable with objects of the parent class without altering the correctness of the program. If a subclass cannot honestly fulfil the parent's contract, the inheritance relationship is wrong.

Violation — Penguin breaks the Bird contract:

```

class Bird {
    void fly() { ... }
}
class Penguin extends Bird {
    void fly() { throw new UnsupportedOperationException(); }
}

```

Refactored — flight is a separate capability:

```

interface FlyingBird { void fly(); }

class Sparrow implements FlyingBird {
    public void fly() { ... }
}

class Penguin { /* no fly() – and none is expected */ }

```

I — Interface Segregation Principle

Clients should not be forced to depend on methods they do not use. Prefer several small, focused interfaces over one large, general-purpose interface.

Violation — one bloated interface:

```

interface Worker {
    void work();
    void eat(); // irrelevant for some implementers
}

```

Refactored — segregated interfaces:

```

interface Workable { void work(); }
interface Eatable { void eat(); }

class Robot implements Workable {
    public void work() { ... }
}

```

```
class Human implements Workable, Eatable {  
    public void work() { ... }  
    public void eat() { ... }  
}
```

D — Dependency Inversion Principle

High-level modules should not depend on low-level modules; both should depend on abstractions. In practice: depend on interfaces, not on concrete implementations. This is the principle behind Dependency Injection in Spring Boot.

Violation — direct coupling to a concrete class:

```
class OrderService {  
    private MySQLRepository repository = new MySQLRepository();  
}
```

Refactored — depends on an abstraction, injected from outside:

```
class OrderService {  
    private final OrderRepository repository; // interface  
  
    OrderService(OrderRepository repository) { // Dependency Injection  
        this.repository = repository;  
    }  
}
```

This is exactly what happens in Spring when a bean is annotated with `@Autowired`: Spring injects a concrete implementation of an interface at runtime, so the service class itself never depends on a specific implementation.

2. Conclusion

SOLID is not a set of rigid rules but a set of guidelines that, applied with judgement, lead to code that is easier to test, extend, and reason about. Single Responsibility keeps classes focused; Open/Closed lets behaviour grow without destabilising existing code; Liskov Substitution keeps inheritance hierarchies honest; Interface Segregation keeps contracts lean; and Dependency Inversion decouples high-level logic from low-level details. Together, they form the theoretical basis for the layered, interface-driven architecture used throughout modern frameworks such as Spring Boot.

References

Martin, R. C. — "Design Principles and Design Patterns" (objectmentor.com).

Martin, R. C. — "Clean Architecture: A Craftsman's Guide to Software Structure and Design", Prentice Hall, 2017.